

180 天 AI 应用开发路线

每日学习计划 · 备份版

目标：从传统前端开发逐步转向可求职的 AI 应用开发 / AI 全栈开发。
建议每天 2~3 小时；每天时间较少时可按自己的节奏顺延，不必强行追赶。

阶段	天数	核心内容
1	1 – 14	Python 基础与 AI 开发所需 Python 能力
2	15 – 35	FastAPI、HTTP、数据库、认证
3	36 – 65	LLM API、Prompt、Streaming、Tool Calling
4	66 – 100	RAG、Embedding、向量数据库
5	101 – 135	Agent、MCP、LangGraph
6	136 – 180	综合项目、Docker、部署、简历、面试

使用方法

每天按照“学习 → 动手 → 总结”完成。每 7 天进行一次复盘；每个阶段尽量留下一个可运行的小成果。

阶段一：Python 基础

Day 1 – Day 14

Day 1 | Python 环境与基础语法：安装/确认 Python；变量、数据类型、print、input；完成基础练习。

Day 2 | 条件判断与比较：if/elif/else、比较运算、逻辑运算；完成成绩判断、登录判断。

Day 3 | 循环：for、while、range、break、continue；练习批量处理数据。

Day 4 | 函数：函数定义、参数、返回值、作用域；理解代码复用。

Day 5 | 文件、JSON、异常处理、模块：文件读写、JSON load/dump、try/except、模块导入；完成 JSON 数据处理。

Day 6 | dict/list 强化：空 dict/list 判断、遍历、增删改查、嵌套结构；按城市筛选用户。

Day 7 | 列表与字典综合：列表/字典推导式；统计城市用户数量；整理代码。

Day 8 | 字符串与内置函数：split、join、replace、strip、sorted、enumerate、zip。

Day 9 | 面向对象基础：class、对象、属性、方法、__init__；把用户/设备抽象成类。

Day 10 | 面向对象进阶：继承、组合、property；设计 Device 类。

Day 11 | venv 与 pip：虚拟环境、pip、requirements.txt；理解依赖隔离。

Day 12 | HTTP 与 requests：GET/POST、params、headers、JSON；调用公开 API。

Day 13 | Python 小项目：完成用户查询工具：API + 城市筛选 + 异常处理 + 格式化输出。

Day 14 | Python 阶段复盘：整理知识点；独立重写小项目，确保基础能力扎实。

阶段二：FastAPI + HTTP + 数据库

Day 15 – Day 35

Day 15 | FastAPI 入门：安装 FastAPI/Uvicorn；创建第一个 API；理解路由和响应。

Day 16 | GET / POST：Path、Query、Request Body、Pydantic。

Day 17 | 参数校验：必填/可选字段、类型校验、默认值；理解 422。

Day 18 | Swagger / OpenAPI：使用接口文档测试 API，理解 schema。

Day 19 | CRUD API：设计用户增删改查，先用内存数据实现。

Day 20 | HTTP 深入：状态码、Header、Cookie、Token。

Day 21 | CORS 与前后端联调：配置 CORS；让前端调用 FastAPI。

Day 22 | 数据库基础：关系型数据库、表、主键、外键；认识 SQLite。

Day 23 | SQL 基础：SELECT/INSERT/UPDATE/DELETE、WHERE、ORDER BY、JOIN。

Day 24 | SQLAlchemy：模型、Session、查询；连接 SQLite。

Day 25 | 数据库设计：用户、会话、文档等表；理解关系。

Day 26 | FastAPI + 数据库：完成持久化 CRUD 与异常处理。

Day 27 | 认证基础：密码哈希、登录流程、JWT 思想。

Day 28 | JWT 实战：登录、Token 验证、受保护接口。

Day 29 | 依赖注入：FastAPI Depends；抽取数据库 Session、当前用户。

Day 30 | 文件上传：UploadFile；为后续 RAG 做准备。

Day 31 | 后台任务：BackgroundTasks；理解异步与后台任务。

Day 32 | async / await：协程与 I/O；理解异步场景。

Day 33 | 日志与配置：环境变量、logging；避免密钥写死。

Day 34 | 测试：pytest、API 测试；编写基础测试。

Day 35 | 阶段项目：完成用户 + 登录 + 数据库 + 文件上传的 FastAPI 后端。

阶段三：LLM API + Prompt + Streaming + Tool Calling

Day 36 – Day 65

- Day 36 | 认识 LLM：模型、Token、上下文窗口、temperature。
- Day 37 | 第一个 LLM API：配置 API Key；Python 调用模型。
- Day 38 | Prompt 基础：system/user、约束、输出格式。
- Day 39 | 结构化输出：让模型稳定输出 JSON，并用 Python 校验。
- Day 40 | Prompt 模板：变量化 Prompt，建立复用模板。
- Day 41 | Few-shot：用示例约束输出。
- Day 42 | 模型参数：理解 temperature、max tokens 等。
- Day 43 | 上下文管理：多轮消息、历史记录、上下文长度。
- Day 44 | Streaming：理解流式输出；逐步返回模型内容。
- Day 45 | FastAPI + LLM：把 LLM 调用封装成 API。
- Day 46 | 前端聊天：让前端连接聊天 API，显示流式消息。
- Day 47 | Token 与成本：理解 Token 计费与成本控制。
- Day 48 | Prompt 调试：建立测试集，比较准确性和稳定性。
- Day 49 | System Prompt：设计角色、规则和边界。
- Day 50 | Tool Calling 概念：理解模型提出工具调用、程序执行、结果回传。
- Day 51 | Tool Calling 实战 1：实现计算器等简单工具。
- Day 52 | Tool Calling 实战 2：多个工具注册、参数校验。
- Day 53 | Tool Calling + FastAPI：建立统一工具接口。
- Day 54 | 外部 API 工具：让模型调用 HTTP API，处理超时异常。
- Day 55 | 数据库工具：把数据库查询包装成 Tool。
- Day 56 | 工具安全：白名单、参数校验、权限和确认机制。
- Day 57 | LLM 应用架构：Controller / Service / LLM / Tool 分层。
- Day 58 | 对话记忆：短期记忆、历史消息、摘要。
- Day 59 | Prompt Injection：理解提示注入与基础防护。
- Day 60 | 错误处理：限流、超时、重试、fallback。
- Day 61 | LLM 小项目：完成 AI 助手：聊天 + Streaming + Tool Calling。
- Day 62 | 项目重构：整理目录、配置、日志、测试、README。
- Day 63 | 阶段复盘：回顾 LLM API、Prompt、Streaming、Tool Calling。
- Day 64 | 综合练习：从零实现一个 AI 客服/信息查询助手。
- Day 65 | 阶段总结：形成 LLM 应用开发知识地图。

阶段四：RAG + Embedding + 向量数据库

Day 66 – Day 100

Day 66 | RAG 是什么：理解 RAG 与模型参数知识的区别。

Day 67 | 文档处理：读取 TXT/Markdown，统一文本格式。

Day 68 | PDF 解析：提取 PDF 文本、标题、页码等元数据。

Day 69 | Chunking：固定长度与按段落切分。

Day 70 | Chunk 优化：overlap、标题层级、语义边界。

Day 71 | Embedding：理解向量、Embedding、相似度。

Day 72 | 向量相似度：余弦相似度；实现简单向量检索。

Day 73 | 向量数据库：建立本地向量索引并保存 Embedding。

Day 74 | 向量检索：Top-K 检索。

Day 75 | 最小 RAG：问题 → Embedding → 检索 → Prompt → LLM。

Day 76 | RAG Prompt：把检索结果注入 Prompt。

Day 77 | 引用与溯源：回答附来源文件/页码。

Day 78 | RAG API：用 FastAPI 封装上传与问答。

Day 79 | 文档上传：多文件上传、解析、切分、入库。

Day 80 | Metadata：保存文件名、页码、标题等。

Day 81 | 过滤检索：按用户、文档、类型过滤。

Day 82 | 混合检索：关键词检索 + 向量检索。

Day 83 | Rerank：理解重排序并比较效果。

Day 84 | 多查询：Query expansion 提高召回。

Day 85 | 上下文压缩：减少无关片段与 Token。

Day 86 | RAG 评估：建立问题集，评估召回与回答。

Day 87 | 幻觉分析：区分检索错误与生成错误。

Day 88 | RAG 安全：权限、提示注入、恶意文档。

Day 89 | 知识库管理：新增、删除、重新索引。

Day 90 | 多用户知识库：实现用户级知识隔离。

Day 91 | 聊天记忆 + RAG：实现带历史上下文的问答。

Day 92 | RAG Streaming：检索完成后流式输出。

Day 93 | RAG 项目 1：开始个人知识库 AI 助手。

Day 94 | RAG 项目 2：完成上传、解析、切分、Embedding。

Day 95 | RAG 项目 3：完成检索、回答、引用、Streaming。

Day 96 | RAG 项目 4：加入 metadata 过滤。

Day 97 | RAG 项目 5：加入评估数据集。

Day 98 | RAG 项目 6：优化 chunk、Top-K、Prompt。

Day 99 | RAG 项目 7：优化异常、日志、成本。

Day 100 | RAG 项目 8：整理项目结构与 README。

阶段五：Agent + MCP + LangGraph

Day 101 – Day 135

Day 101 | Agent 概念：理解 Workflow 与 Agent 的区别。

Day 102 | Agent Loop：实现 Thought/Action/Observation 简化流程。

Day 103 | 工具注册：统一 Tool schema。

Day 104 | Agent 状态：保存任务状态、消息和工具结果。

Day 105 | 规划与执行：把复杂任务拆成步骤。

Day 106 | 错误恢复：工具失败后的重试与替代。

Day 107 | Agent 记忆：短期状态与长期记忆。

Day 108 | Agent 安全：限制工具权限、循环次数和预算。

Day 109 | LangGraph 入门：State、Node、Edge、Graph。

Day 110 | 第一个 LangGraph：实现状态图与条件分支。

Day 111 | LangGraph Agent：把 Tool Calling 接入图。

Day 112 | 循环 Agent：判断 → 工具 → 观察 → 继续/结束。

Day 113 | Human in the loop：高风险操作前人工确认。

Day 114 | 持久化状态：保存 Agent 状态并支持恢复。

Day 115 | Agent 调试：记录节点输入输出与执行轨迹。

Day 116 | 多 Agent 概念：supervisor、worker 等模式。

Day 117 | 多 Agent 实战：不同 Agent 分工完成任务。

Day 118 | Agent + RAG：Agent 自主决定何时查询知识库。

Day 119 | Agent + API：组合多个外部工具。

Day 120 | Agent 评估：建立任务集，评估成功率。

Day 121 | MCP 是什么：理解 MCP Server/Client/Tool/Resource。

Day 122 | MCP Server：搭建简单 MCP Server。

Day 123 | MCP Client：让应用连接 MCP Server。

Day 124 | MCP + 数据库：暴露安全的数据查询工具。

Day 125 | MCP + 文件：让 Agent 使用文件资源。

Day 126 | MCP 安全：权限、输入校验、工具边界。

Day 127 | Agent API：用 FastAPI 提供 Agent 服务。

Day 128 | Agent 前端：显示 Agent 工具调用状态。

Day 129 | Agent 项目 1：设计 AI 工作助手架构。

Day 130 | Agent 项目 2：实现状态图、工具系统、记忆。

Day 131 | Agent 项目 3：加入 RAG 与 MCP。

Day 132 | Agent 项目 4：加入人工确认与安全限制。

Day 133 | Agent 项目 5：日志、持久化、错误恢复。

Day 134 | Agent 项目 6：测试复杂任务与边界情况。

Day 135 | 阶段复盘：总结 Agent、LangGraph、MCP。

阶段六：综合项目 + Docker + 部署 + 求职

Day 136 – Day 180

Day 136 | 最终项目选题：确定最终作品：AI 知识库/企业 AI 助手/AI 工作台。

Day 137 | 需求分析：功能清单、非功能需求、用户流程。

Day 138 | 系统架构：前端、FastAPI、LLM、RAG、Agent、数据库。

Day 139 | 项目初始化：Git、venv、依赖、目录结构。

Day 140 | 后端骨架：FastAPI、配置、日志、异常处理。

Day 141 | 数据库设计：用户、会话、文档、消息等表。

Day 142 | 认证：注册、登录、JWT。

Day 143 | 前端骨架：聊天、知识库、设置页面。

Day 144 | LLM 服务层：统一模型调用、Streaming、错误处理。

Day 145 | 聊天功能：基础对话与历史记录。

Day 146 | 知识库上传：上传与解析。

Day 147 | RAG Pipeline：切分、Embedding、入库。

Day 148 | 检索问答：Top-K 检索与回答。

Day 149 | 引用展示：前端显示来源文件/页码。

Day 150 | Agent 功能：加入工具调用与任务执行。

Day 151 | MCP 接入：接入 MCP Server/工具。

Day 152 | 权限：用户数据隔离与工具权限。

Day 153 | 任务状态：长任务状态与执行日志。

Day 154 | UI 优化：Loading、错误提示、聊天体验。

Day 155 | 测试 1：单元测试、API 测试。

Day 156 | 测试 2：RAG 与 Agent 场景测试。

Day 157 | 性能检查：检查数据库、检索、LLM 耗时。

Day 158 | 成本优化：Token、缓存、模型选择。

Day 159 | 安全检查：密钥、CORS、输入、文件、Prompt Injection。

Day 160 | Docker 基础：镜像、容器、Dockerfile。

Day 161 | Dockerize 后端：FastAPI 容器化。

Day 162 | Docker Compose：编排后端、数据库等服务。

Day 163 | 生产配置：环境变量、配置文件、日志。

Day 164 | 部署准备：服务器、域名、端口规划。

Day 165 | Linux 基础：命令、进程、端口、权限。

Day 166 | 部署后端：Docker 项目部署。

Day 167 | 反向代理：Nginx、HTTPS、域名。

Day 168 | 部署前端：构建并通过 Nginx 提供服务。

Day 169 | 上线测试：注册 → 上传 → 检索 → 问答 → Agent。

Day 170 | 监控与日志：服务日志、错误、资源使用。

Day 171 | 故障排查：模拟 API 超时、服务重启、数据库异常。

Day 172 | 项目打磨 1：README、架构图、功能截图。

Day 173 | 项目打磨 2：整理核心代码。

Day 174 | 项目打磨 3：增加 AI 亮点功能。

Day 175 | 项目打磨 4：部署文档与环境变量说明。

Day 176 | 简历准备 1：把项目写成简历项目经历。

Day 177 | 简历准备 2：整理 AI 应用岗位技术关键词。

Day 178 | 面试准备 1：Python、FastAPI、HTTP、数据库。

Day 179 | 面试准备 2：LLM、Prompt、Streaming、Tool Calling。

Day 180 | 面试准备 3：RAG、Embedding、向量数据库、Rerank。

180 天完成后的能力目标

- 能够使用 Python 独立完成 AI 应用后端开发。
- 能够使用 FastAPI 开发真实 API，并完成数据库、认证、文件上传等常见后端能力。
- 能够调用 LLM API，掌握 Prompt、Streaming、结构化输出和 Tool Calling。
- 能够独立搭建 RAG：文档解析、Chunk、Embedding、向量检索、引用与评估。
- 理解并能够使用 Agent、LangGraph、MCP，完成工具型 AI 应用。
- 能够把完整 AI 应用 Docker 化并部署上线。
- 最终形成至少 1 个可以放进简历、可以演示、可以在面试中完整讲清楚的 AI 应用项目。

备份说明

这份版本保留了前面已经确定的学习方向，包括 Day 5 的文件、JSON、异常处理、模块导入，以及 Day 6 的 dict、list 等 Python 基础内容。后续学习可以直接按 Day 编号继续。